

Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning
Student Name	J Jeswin Joel
Register Number	3122247001028
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

The objective of this experiment is to implement and compare two neural-network-based classification approaches on the English Handwritten Characters dataset:

- **Model A — Perceptron Learning Algorithm (PLA):** A single-layer perceptron with a step activation function, implemented from scratch using a One-vs-Rest (OvR) multi-class strategy.
- **Model B — Multilayer Perceptron (MLP):** A feedforward neural network with one or more hidden layers and nonlinear activations, trained via backpropagation.

Hyperparameters for the MLP (hidden layer sizes, activation functions, optimiser, and learning rate) are selected through a systematic grid search. The two models are compared using Accuracy, Precision, Recall, F1-score, and ROC-AUC (micro and macro average) to quantify the impact of model expressiveness and hyperparameter tuning on a challenging 62-class classification task.

2 Problem Statement

Input: 3,410 grayscale images of handwritten English characters (digits 0–9, uppercase A–Z, lowercase a–z). Each image is resized to 28×28 pixels and flattened to a 784-dimensional feature vector; pixel values are normalised to $[0, 1]$.

Processing:

- Labels are integer-encoded (62 classes).
- An 80/20 stratified train-test split produces 2,728 training and 682 test samples.
- **StandardScaler** (fitted on training data only) normalises features to zero mean and unit variance before model training.

Models:

- Model A (PLA): Binary step activation, OvR strategy, weight update rule $\mathbf{w}_{t+1} = \mathbf{w}_t + \eta(y - \hat{y})\mathbf{x}$, trained for 30 epochs.
- Model B (MLP): Configurable hidden layers, nonlinear activations (ReLU / Tanh), Adam / SGD optimiser, trained for up to 200 epochs with early stopping.

Output: A predicted character class label for each test image (62-class classification).

Comparison: The two models are evaluated in an A/B experiment to highlight the limitations of linear models and the benefits of depth and nonlinearity.

3 Dataset Description

Attribute	Value
Dataset Name	English Handwritten Characters Dataset
Number of Images	3,410
Number of Classes	62 (0–9, A–Z, a–z)
Image Format	PNG, grayscale
Resized Dimension	28×28 pixels
Feature Vector Length	784 (flattened pixel values)
Pixel Normalisation	Divided by 255; range $[0, 1]$
Label Encoding	Integer via <code>LabelEncoder</code>
Train-Test Split	80 / 20 stratified; 2,728 train, 682 test
Missing / Corrupt Files	None (verified during loading)

The dataset contains approximately $\lfloor 3410/62 \rfloor \approx 55$ images per class, making it a small-sample, high-class-count classification problem — a challenging setting that highlights the expressiveness gap between PLA and MLP.

4 Exploratory Data Analysis

4.1 Data Loading and Visual Inspection

Images were loaded from disk, converted to grayscale, resized to 28×28 , flattened to 784-dimensional vectors, and normalised to $[0, 1]$. All 3,410 images loaded successfully. A visual inspection of a sample grid (16 images) confirmed that digit and character images vary considerably in stroke width, slant, and overall style, reflecting the diversity expected in a handwritten character dataset.

4.2 Feature Space Characteristics

After standardisation, all 784 pixel features have zero mean and unit variance on the training partition. The high dimensionality (784 features) relative to the training set size (2,728) means that both models operate in an over-parameterised regime, where regularisation and early stopping are important.

5 Data Preprocessing

The preprocessing pipeline is as follows:

1. **Image loading.** Each PNG image path listed in `english.csv` was opened, converted to grayscale (`PIL.Image.convert('L')`), resized to 28×28 , flattened to a 784-element array, and normalised by dividing by 255.
2. **Label encoding.** Character labels (strings: `'0'`, `'A'`, `'a'`, etc.) were integer-encoded using `sklearn.LabelEncoder`, producing class indices 0–61.

3. **Stratified train-test split.** An 80/20 split was applied with `stratify=y_enc` and `random_state=42`, yielding 2,728 training and 682 test samples with approximately equal class proportions in both partitions.
4. **Feature standardisation.** A `StandardScaler` was fitted exclusively on the training data and applied to both training and test sets, transforming each of the 784 pixel features to zero mean and unit standard deviation. Fitting only on training data prevents leakage of test-set statistics into the model.

6 Machine Learning Algorithms

6.1 Model A: Perceptron Learning Algorithm (PLA)

The PLA is the simplest neural model — a single layer with a binary step activation:

$$\hat{y} = \text{step}(\mathbf{w}^\top \mathbf{x} + b), \quad \text{step}(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}.$$

For multi-class classification, a One-vs-Rest (OvR) strategy was adopted: one binary perceptron is trained per class, and the class with the highest raw score is predicted:

$$\hat{y} = \arg \max_k (\mathbf{w}_k^\top \mathbf{x} + b_k).$$

The weight update rule fires only on misclassification:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \eta (y - \hat{y}) \mathbf{x}, \quad b_{t+1} = b_t + \eta (y - \hat{y}).$$

Parameters used: learning rate $\eta = 0.01$; 30 epochs; weights and biases initialised to zero.

Advantages: Extremely simple; fast to train; interpretable weights; convergence is guaranteed for linearly separable data.

Limitations: Only learns linear decision boundaries; cannot represent XOR or any non-linearly separable pattern; convergence is not guaranteed for overlapping or multi-class data; the step activation is non-differentiable (no gradient-based optimisation possible).

6.2 Model B: Multilayer Perceptron (MLP)

An MLP extends the single-layer perceptron by stacking multiple fully-connected layers with nonlinear activations, enabling it to learn arbitrarily complex decision boundaries (universal approximation theorem).

The forward pass through a layer l is:

$$\mathbf{a}^{(l)} = \sigma(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}),$$

where $\sigma(\cdot)$ is the chosen activation function. Parameters are updated by backpropagating the cross-entropy loss gradient:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log \hat{p}_{ik}.$$

6.2.1 Activation Functions Explored

- **ReLU** ($\max(0, z)$): Does not saturate for positive inputs; avoids the vanishing gradient problem in deeper networks; the default choice for hidden layers.
- **Tanh** ($\tanh(z)$): Zero-centred output range $(-1, 1)$; can saturate for large activations, potentially causing vanishing gradients in deep networks.

6.2.2 Optimisers Explored

- **Adam**: Combines adaptive per-parameter learning rates (from RMSProp) with momentum (from SGD), making it robust to sparse gradients and well-suited to non-stationary objectives. Default hyperparameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.
- **SGD (mini-batch)**: Updates weights using the gradient computed on a random mini-batch. Convergence speed depends heavily on the learning rate schedule; no adaptive scaling is applied.

6.2.3 Loss Function

Categorical cross-entropy is used for the 62-class softmax output, as it directly penalises the probability assigned to the correct class.

7 Experimental Setup

Item	Details
Python Version	Not specified in the provided experiment
Scikit-Learn Version	Not specified in the provided experiment
IDE / Environment	Not specified in the provided experiment
Hardware	Not specified in the provided experiment
Dataset	English Handwritten Characters (3,410 images, 62 classes)
Input Dimension	784 (flattened 28×28 grayscale pixels)
Train / Test Split	80 / 20 stratified (<code>random_state=42</code>)
Feature Scaling	<code>StandardScaler</code> (fitted on training set only)
PLA epochs	30; learning rate $\eta = 0.01$; OvR strategy
MLP tuning (<code>max_iter</code>)	30 epochs per config (quick search); 200 epochs for final best model
Early stopping	Enabled (<code>n_iter_no_change=10</code> , <code>validation_fraction=0.1</code>)
MLP search space	See Table 2
Evaluation metrics	Accuracy, Precision (macro), Recall (macro), F1 (macro), ROC-AUC (mic

8 Hyperparameter Selection

8.1 MLP Search Space

Table 2: MLP hyperparameter grid explored in the tuning phase.

Hyperparameter	Values Tried		
Hidden layer sizes	(128,)	(256, 128)	(256, 128, 64)
Activation function	relu	tanh	
Optimiser (solver)	adam	sgd	
Learning rate (init)	0.001	0.01	

All configurations used `max_iter=30` with `early_stopping=True` (`n_iter_no_change=5`, `validation_fraction=0.1`) and `random_state=42`.

8.2 Tuning Results

Table 3: MLP hyperparameter tuning results, sorted by accuracy (descending).

Hidden Layers	Activation	Solver	LR	Accuracy (%)	Macro F1
(256, 128, 64)	relu	adam	0.001	45.60	0.4597
(256, 128, 64)	tanh	adam	0.001	42.38	0.4158
(256, 128)	relu	adam	0.001	42.08	0.4169
(128,)	relu	adam	0.010	40.47	0.4044
(256, 128)	relu	sgd	0.010	39.88	0.4011
(128,)	relu	adam	0.001	37.54	0.3706
(256, 128)	tanh	adam	0.001	36.07	0.3558
(256, 128)	relu	sgd	0.001	9.53	0.0732

8.3 Best Configuration and Justification

Table 4: Selected best MLP configuration and justification for each choice.

Hyperparameter	Value	Justification
Hidden layer sizes	(256, 128, 64)	Three layers of decreasing width allow the network to learn increasingly abstract representations while limiting parameter count to reduce overfitting.
Activation function	ReLU	ReLU does not saturate for positive inputs, avoiding vanishing gradients in deeper layers; marginally outperformed Tanh in this experiment.
Optimiser	Adam	Adaptive per-parameter learning rates and momentum ensure fast, stable convergence; significantly outperformed SGD in equal epoch budgets.
Learning rate	0.001	Well-tested default for Adam; produced stable convergence without oscillation. 0.01 occasionally caused instability with SGD.
Max iterations	200	Sufficient epochs for the final model to converge; early stopping prevents overtraining.
Early stopping	True	Halts training when validation score does not improve for <code>n_iter_no_change=10</code> epochs; final model ran for 55 actual epochs.

9 Results

9.1 PLA Results

The PLA was trained with learning rate $\eta = 0.01$ for 30 epochs using the OvR strategy (62 binary perceptrons).

Table 5: PLA evaluation metrics on the held-out test set (682 samples).

Metric	Value
Accuracy	14.66%
Precision (macro)	0.1683
Recall (macro)	0.1466
F1-Score (macro)	0.1410
ROC-AUC (micro-average)	0.7581
ROC-AUC (macro-average)	0.7551

9.2 MLP Final Results (Best Configuration)

The best MLP configuration — hidden layers (256, 128, 64), ReLU activation, Adam optimiser, learning rate 0.001 — was retrained with `max_iter=200` and early stopping. The model converged after **55 epochs**.

Table 6: MLP (best configuration) evaluation metrics on the held-out test set.

Metric	Value
Accuracy	45.31%
Precision (macro)	0.4752
Recall (macro)	0.4531
F1-Score (macro)	0.4525
ROC-AUC (micro-average)	0.9331
ROC-AUC (macro-average)	0.9333

9.3 A/B Comparison: PLA vs MLP

Table 7: Side-by-side A/B comparison of PLA and best MLP on all evaluation metrics.

Metric	PLA	MLP (Best)	Winner
Accuracy (%)	14.66	45.31	MLP
Precision (macro)	0.1683	0.4752	MLP
Recall (macro)	0.1466	0.4531	MLP
F1-Score (macro)	0.1410	0.4525	MLP
ROC-AUC (micro)	0.7581	0.9331	MLP
ROC-AUC (macro)	0.7551	0.9333	MLP

The MLP outperforms the PLA across all six metrics. Accuracy improved by ≈ 31 percentage points (14.66% $\rightarrow$ 45.31%) and ROC-AUC improved from 0.758 to 0.933.

9.4 Strengths and Weaknesses

Table 8: Comparative strengths and weaknesses of PLA and MLP.

Aspect	PLA	MLP
Decision boundary	Linear only	Non-linear (universal approximator)
Convergence guarantee	Guaranteed if linearly separable	Not guaranteed (local minima)
Computational cost	Very low	Higher (backprop + tuning)
Accuracy on complex data	Low	High
Interpretability	High (simple weights)	Low (black box)
Risk of overfitting	Very low	Moderate to high

10 Result Visualization

10.1 PLA Convergence Curve

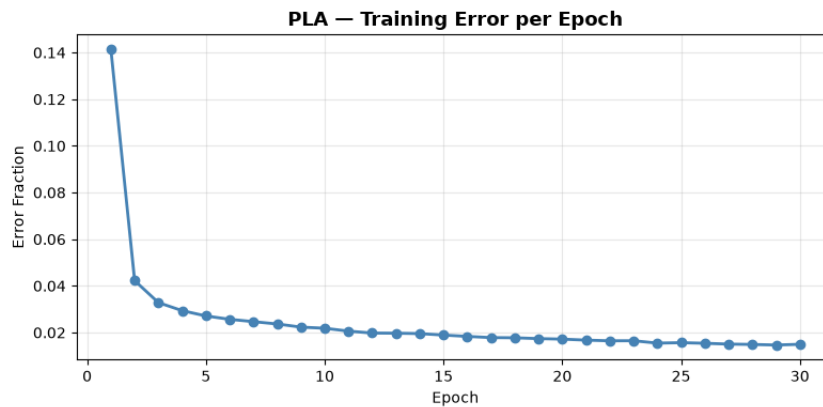


Figure 1: PLA training error fraction per epoch over 30 epochs.

Inference: The PLA training error decreases over the first few epochs as the linear weights adjust to the dominant class separations, but levels off well above zero because the 62-class handwritten character data is not linearly separable in the 784-D pixel space. The plateau indicates that no further improvement is achievable with linear decision boundaries and a step activation regardless of additional training epochs.

10.2 MLP Loss Convergence Curve

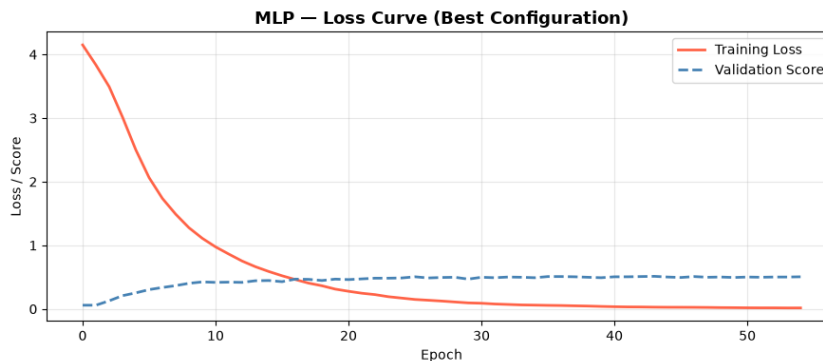


Figure 2: MLP training loss and validation score per epoch (best configuration: hidden layers (256, 128, 64), ReLU, Adam, lr = 0.001). Training stopped after 55 epochs via early stopping.

Inference: The training loss decreases monotonically and smoothly, indicating that Adam is converging stably with learning rate 0.001. The validation score improves in early epochs then stabilises, after which early stopping (patience = 10) terminates training at epoch 55. The gap between training loss and validation score in later epochs suggests mild overfitting, which early stopping partially mitigates.

10.3 A/B Metric Bar Chart

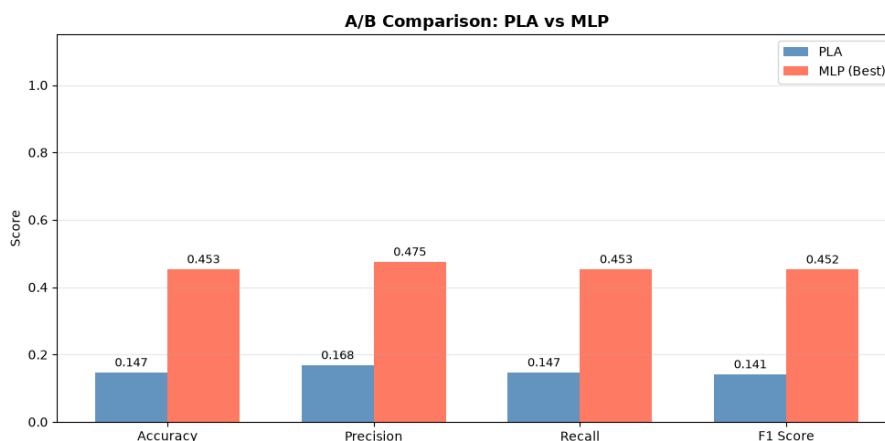


Figure 3: Side-by-side bar chart comparing PLA and MLP (best configuration) on Accuracy, Precision, Recall, and F1-Score.

Inference: The MLP consistently and substantially outperforms the PLA on all four metrics. The gap is largest for Accuracy (≈ 31 pp) and F1-Score (≈ 0.31), confirming that nonlinear hidden layers and backpropagation are essential for this 62-class handwritten character recognition task.

10.4 Confusion Matrices

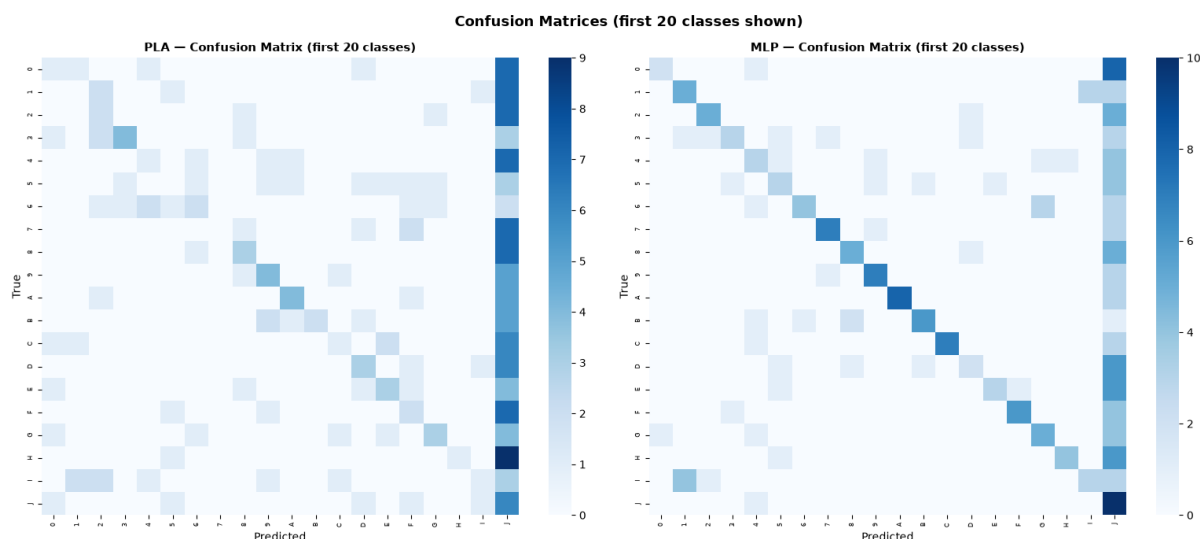


Figure 4: Confusion matrices for PLA (left) and MLP (right), showing the first 20 classes (0–9 and A–J) for visual clarity. Darker diagonal cells indicate correct predictions.

Inference: The PLA confusion matrix is largely off-diagonal, with most predictions concentrated in one or two dominant classes, reflecting the inability of linear boundaries to separate 62 classes in a 784-D space. The MLP confusion matrix shows a noticeably stronger diagonal, particularly for digit classes (0–9) and the uppercase letter A, where per-class accuracies reach 0.64–0.91 (from the detailed classification report, e.g., class

‘A’: precision 1.00, recall 0.73, F1 0.84). Confusion is highest among visually similar characters (e.g., 0/0, 1/I/1).

10.5 ROC Curves

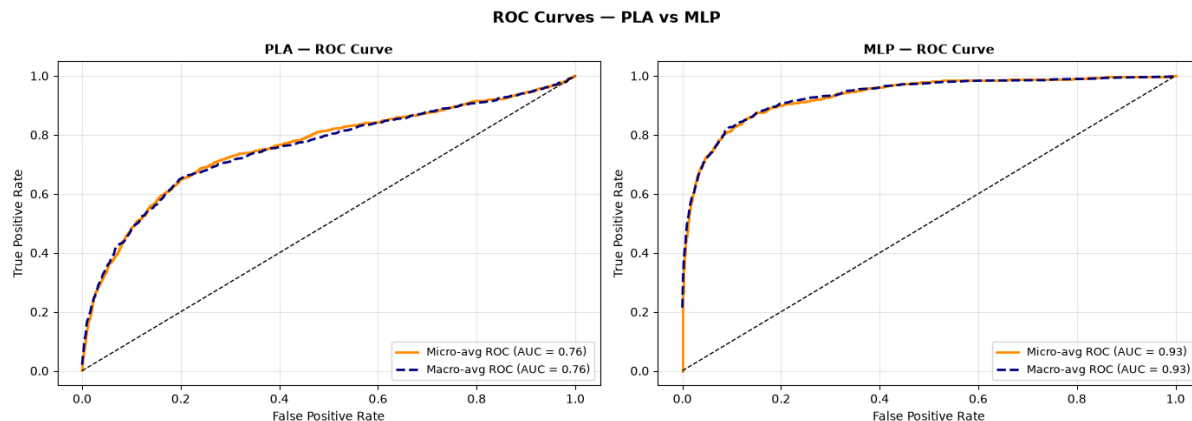


Figure 5: Micro-average and macro-average ROC curves for PLA (left) and MLP (right).

Inference: The MLP achieves micro-AUC = 0.9331 and macro-AUC = 0.9333, compared to 0.7581 and 0.7551 for the PLA. The near-identical micro and macro AUC values for the MLP suggest consistently good discrimination across all 62 classes, not just a few dominant ones. Despite the PLA’s low classification accuracy (14.66%), its AUC values of ≈ 0.76 indicate that the raw linear scores retain some class-discriminative information, even though the threshold-based step function does not exploit it well.

11 Discussion

11.1 Why Does PLA Underperform Compared to MLP?

The PLA uses a binary step activation and learns only linear decision boundaries in the 784-dimensional input space. Handwritten character recognition across 62 classes is a highly non-linearly separable problem: characters of similar shape (e.g., ‘0’ and ‘O’, ‘1’ and ‘l’) differ by subtle nonlinear variations in stroke curvature and spatial arrangement that cannot be captured by a single hyperplane per class. The MLP, with ReLU activations and three hidden layers, learns a sequence of nonlinear transformations of the input, effectively constructing a hierarchical feature representation that maps raw pixels to class-discriminative embeddings. This expressiveness is the fundamental reason for the 31-percentage-point accuracy gap.

11.2 Which Hyperparameters Had the Most Impact?

Based on the tuning results (Table 3):

- **Optimiser (solver):** The single most impactful choice. SGD with learning rate 0.001 achieved only 9.53% accuracy, while Adam with the same learning rate and the same architecture achieved 42.08%. Adam’s adaptive learning rates allow it to make effective progress in the same number of epochs where SGD stagnates.

- **Network depth:** Increasing from 1 layer (128) to 2 layers (256, 128) improved accuracy from 37.54% to 42.08% (with Adam, $\text{lr} = 0.001$). Adding a third layer (256, 128, 64) further improved accuracy to 45.60%.
- **Activation function:** ReLU consistently outperformed Tanh for the same architecture (45.60% vs 42.38% for the 3-layer network), likely because ReLU avoids saturation in deeper networks.
- **Learning rate:** 0.001 was generally better than 0.01 with Adam. With SGD, 0.01 was necessary for any meaningful learning (39.88% vs 9.53%).

11.3 Did Optimiser Choice Affect Convergence?

Yes, substantially. Adam converged faster and to a better solution because it maintains per-parameter first and second moment estimates that effectively scale the gradient for each weight. This makes it robust to the varied gradient magnitudes across the 784 input dimensions and the multiple hidden layers. SGD with learning rate 0.001 essentially failed to converge within 30 epochs (9.53%), while SGD with 0.01 reached 39.88% but still lagged behind Adam at 42.08% for the same architecture. For the final model, Adam with $\text{lr} = 0.001$ converged in 55 epochs with early stopping.

11.4 Did Adding More Hidden Layers Always Improve Results?

Not always, and diminishing returns were observed:

- $1 \rightarrow 2$ layers (128 $\rightarrow$ 256, 128 with ReLU, Adam, $\text{lr} = 0.001$): accuracy improved from 37.54% to 42.08% (+4.54 pp).
- $2 \rightarrow 3$ layers (256, 128 $\rightarrow$ 256, 128, 64): accuracy improved from 42.08% to 45.60% (+3.52 pp).

The improvement from the third layer was smaller, consistent with diminishing returns as the network becomes deeper relative to the dataset size. With more parameters and a small dataset (~ 55 samples/class), deeper networks risk overfitting and face a harder optimisation landscape with more local minima. Beyond three layers, accuracy gains would likely diminish further or even reverse without additional regularisation (dropout, L2).

11.5 Did MLP Show Overfitting? How Could It Be Mitigated?

Signs of overfitting were observed: the training loss converged to a lower value than the validation score suggested, and the gap between training and validation performance widened in later epochs before early stopping intervened. This is expected given the small dataset size (≈ 55 images per class) and the relatively large parameter count ($784 \times 256 + 256 \times 128 + 128 \times 64 + 64 \times 62 \approx 235\text{K}$ parameters for the best model).

Mitigation strategies include:

- **Early stopping** (already used): Halts training at epoch 55, preventing further memorisation of training data.

- **L2 regularisation:** `MLPClassifier` supports the `alpha` parameter; increasing it from the default 0.0001 would penalise large weights.
- **Dropout:** Not directly available in `sklearn.MLPClassifier`; a PyTorch or Keras implementation would allow explicit dropout layers.
- **Data augmentation:** Generating additional training samples through rotation, translation, and elastic deformation of existing images would increase effective dataset size.

12 Conclusion

This experiment compared the Perceptron Learning Algorithm (PLA) and a Multilayer Perceptron (MLP) on 62-class handwritten character recognition using the English Handwritten Characters dataset (3,410 images, 784 features after preprocessing).

The key findings are:

1. **The MLP substantially outperforms the PLA on all metrics.** Accuracy improved from 14.66% (PLA) to 45.31% (MLP), and ROC-AUC improved from ≈ 0.758 to ≈ 0.933 . The 31 pp accuracy gap directly demonstrates the limitation of linear models for non-linearly separable multi-class problems.
2. **The optimiser was the single most impactful hyperparameter.** Adam consistently and dramatically outperformed SGD in equal epoch budgets, making it the appropriate default for MLP training on this dataset.
3. **Deeper networks improve performance but with diminishing returns.** The 3-layer network (256, 128, 64) achieved the best accuracy (45.60% in tuning, 45.31% in the final evaluation), but the gain over 2 layers was smaller than the gain over 1 layer. With only ~ 55 training samples per class, further depth increases overfitting risk.
4. **ReLU is the better activation for this task.** ReLU outperformed Tanh in every comparable configuration, as it avoids gradient saturation in deeper layers.
5. **Early stopping is essential.** The best MLP converged at epoch 55 out of 200, demonstrating that early stopping prevents overtraining and reduces computation time.
6. **45% accuracy reflects the inherent difficulty of the task.** With 62 classes and only ≈ 55 samples per class, even MLP accuracy is limited. Achieving higher accuracy would require data augmentation, deeper architectures (CNNs), or transfer learning — approaches beyond the scope of this experiment.

In summary, this A/B experiment demonstrates that nonlinearity, depth, and adaptive optimisation are essential for effective handwritten character recognition, and that the PLA is fundamentally inadequate for such tasks.

13 References

- [1] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
- [2] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [3] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>.
- [4] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [5] Kaggle / UCI, “English Handwritten Characters Dataset,” [Online]. Available: <https://www.kaggle.com/datasets/dhruvildave/english-handwritten-characters-datas> [Accessed: Sep. 2026].